

Pirate's Hangman

Documentation technique du projet

Jeu du pendu thématique pirate, développé en Java avec JavaFX pour l'interface graphique et MySQL via JDBC pour la persistance du classement.

Projet réalisé dans le cadre de l'ENSA

Sommaire

1. Introduction
2. Architecture du projet (MVC)
3. Documentation des classes
 - 3.1 Package hangman (Main)
 - 3.2 Package hangman.enums (Difficulty)
 - 3.3 Package hangman.db (DBConnection, DictionaryDAO, ScoreDAO)
 - 3.4 Package hangman.models (GameSession, Player, ScoreRecord)
 - 3.5 Package hangman.services (ScoreBoard)
 - 3.6 Package hangman.managers (GameManager, SinglePlayer, Multiplayer)
 - 3.7 Package hangman.ui (GameWindow, BackgroundPane, VirtualKeyboard, MatchEndMenu)
4. Système de score
5. Schéma de la base de données
6. Conclusion

1. Introduction

Pirate's Hangman est une implémentation du jeu classique du Pendu, habillée d'une esthétique de baie pirate. Le joueur tente de retrouver un mot caché en proposant des lettres, avec un nombre limité d'erreurs avant que le pirate ne soit pendu à la potence.

Le projet propose deux modes de jeu :

- **Mode Solo** — un joueur affronte le dictionnaire à une difficulté choisie. Les meilleurs scores sont sauvegardés dans un classement persistant.
- **Mode 1v1 (Duel de Capitaines)** — deux joueurs s'affrontent en alternant le rôle de celui qui choisit le mot secret et celui qui devine. En cas d'égalité, une mort subite départage les capitaines.

Technologies utilisées

- **Java (JDK 17+)** — langage principal du projet.
- **JavaFX** — bibliothèque d'interface graphique.
- **MySQL 8.0+** — base de données relationnelle.
- **JDBC (MySQL Connector/J)** — pilote d'accès aux données.
- **CSS JavaFX** — habillage du thème pirate.

2. Architecture du projet (MVC)

Le projet suit le patron **Modèle-Vue-Contrôleur** avec une séparation claire en couches :

Couche	Packages	Responsabilité
Modèle	hangman.models, hangman.entities	Représente les données du jeu : la session de partie, les joueurs, les mots à deviner.
Accès aux données	hangman.db	Encapsule toutes les requêtes SQL (pattern DAO). Singleton de connexion à la base de données.
Services	hangman.services	Couche métier au-dessus des DAO. Validation, valeurs par défaut, gestion des erreurs.
Contrôleurs	hangman.managers	Orchestrent les manches : <code>SinglePlayer</code> pour le solo, <code>Multiplayer</code> pour le duel.
Vue	hangman.ui	Interface graphique JavaFX : menus, écran de jeu, clavier virtuel, fond d'écran.
Point d'entrée	hangman.Main	Lanceur compatible IDE (VS Code, IntelliJ) avec capture des erreurs.

Flux d'une partie solo

1. L'utilisateur lance le jeu (**Main.main()**) → JavaFX démarre **GameWindow.start()**.
2. Le menu principal s'affiche. L'utilisateur clique sur « Set Sail Alone » et choisit une difficulté.
3. **SinglePlayerManager.startRound()** demande un mot aléatoire à **DictionaryDAO** et crée une **GameSession**.
4. L'écran de jeu s'affiche ; chaque touche pressée appelle **GameSession.guess(c)** et rafraîchit l'interface.
5. À la fin de la manche, le score combiné est calculé. Si le joueur entre dans le top 10, son nom est demandé puis sauvegardé via **ScoreBoard** → **ScoreDAO**.

Flux d'un duel 1v1

1. Configuration : noms des deux capitaines, difficulté, nombre de manches.
2. Pour chaque demi-manche : le « chooser » saisit un mot secret (champ masqué), le « guesser » devine.
3. Score chances-only ajouté au guesser. Inversion des rôles.
4. Quand toutes les manches sont jouées, **determineWinner()** départage.
5. Si égalité : **runTieBreaker()** tire un même mot à la difficulté supérieure pour les deux capitaines. Score complet (chances + temps).

3. Documentation des classes

3.1 Package hangman

Classe : Main

Package : hangman

Rôle : Point d'entrée de l'application. Délibérément ne dérive PAS de **Application** pour pouvoir intercepter une erreur de chargement JavaFX et afficher un message d'aide convivial avec les arguments VM à utiliser.

Méthodes

Signature	Description
<code>public static void main(String[] args)</code>	Lance l'application. Essaie Application.launch(GameWindow.class) . Capture LinkageError et RuntimeException liée à JavaFX, et délègue à printJavaFxHint() .
<code>private static boolean rootCauseIsJavaFxMissing(Throwable t)</code>	Parcourt la chaîne de causes de l'exception et renvoie true si le message contient une signature liée à JavaFX (« JavaFX runtime », etc.).
<code>private static void printJavaFxHint(Throwable err)</code>	Affiche dans stderr les arguments VM à ajouter (--module-path + --add-modules javafx.controls) puis la trace de la pile sous-jacente.

3.2 Package hangman.enums

Classe : Difficulty

Package : hangman.enums

Rôle : Énumération qui définit les paramètres de chaque mode de jeu — le « règlement » du Pendu. Chaque valeur (EASY, MEDIUM, HARD, INSANE) transporte le nombre de chances en solo et l'intervalle de longueur valable pour le mot secret en 1v1.

Attributs

Nom	Type	Rôle
<code>singlePlayerChances</code>	<code>int</code>	Nombre maximal d'erreurs autorisées en mode solo (10, 8, 6, 4).
<code>minLength</code>	<code>int</code>	Longueur minimale du mot secret en mode 1v1.
<code>maxLength</code>	<code>int</code>	Longueur maximale du mot secret en mode 1v1.

Méthodes

Signature	Description
<code>Difficulty(int chances, int min, int max)</code>	Constructeur privé de l'enum.
<code>public int getSinglePlayerChances()</code>	Renvoie le nombre de chances pour cette difficulté.
<code>public int getMinLength()</code>	Renvoie la longueur minimale du mot secret.
<code>public int getMaxLength()</code>	Renvoie la longueur maximale du mot secret.
<code>public Difficulty getNextDifficulty()</code>	Renvoie la difficulté supérieure (utilisée par le mécanisme de mort subite). INSANE renvoie INSANE (déjà au max).

3.3 Package hangman.db

Classe : DBConnection

Package : hangman.db

Rôle : Singleton qui gère la connexion JDBC à MySQL. Le constructeur est privé : les DAO obtiennent la connexion active via **getInstance().getConnection()**, garantissant qu'une seule connexion est ouverte par l'application. Les identifiants sont lus depuis **config.properties**.

Attributs

Nom	Type	Rôle
instance	static DBConnection	Instance unique du singleton.
connection	Connection	Objet de connexion JDBC actif.

Méthodes

Signature	Description
private DBConnection()	Constructeur : charge le pilote MySQL via Class.forName et ouvre la connexion via DriverManager.getConnection .
public static synchronized DBConnection getInstance()	Renvoie l'instance unique (la crée à la première utilisation).
public Connection getConnection()	Renvoie la connexion SQL active utilisée par les DAO.
public void closeConnection()	Ferme la connexion proprement à la fermeture du jeu.
private static Properties loadConfig()	Charge config.properties et applique en surcouche les propriétés système (utile pour les tests).

Classe : DictionaryDAO

Package : hangman.db

Rôle : Data Access Object pour la table **Dictionary**. Encapsule la requête SQL qui récupère un mot aléatoire selon la difficulté.

Méthodes

Signature	Description
public String getRandomWord(Difficulty diff)	Exécute SELECT word FROM Dictionary WHERE difficulty = ? ORDER BY RAND() LIMIT 1 . Renvoie le mot en minuscules ou null si aucun mot pour cette difficulté.

Classe : ScoreDAO

Package : hangman.db

Rôle : Data Access Object pour la table **Leaderboard**. La règle est « plus c'est élevé, mieux c'est » : la colonne **score** est triée DESC pour obtenir le classement.

Méthodes

Signature	Description
<code>public void saveScore(Difficulty d, String name, long score)</code>	Insère une nouvelle ligne dans Leaderboard via PreparedStatement paramétré.
<code>public List<ScoreRecord> getTop10Scores(Difficulty d)</code>	Renvoie jusqu'à 10 lignes triées par score décroissant pour la difficulté donnée.
<code>public boolean isTop10(Difficulty d, long candidate)</code>	True si le score candidat ferait entrer le joueur dans le top 10 (soit moins de 10 lignes, soit candidat > pire qualifié).

3.4 Package hangman.models

Classe : GameSession

Package : hangman.models

Rôle : Moteur générique d'une manche du Pendu. Volontairement « bête » : ne sait pas si elle est solo ou multijoueur. Reçoit un mot et un nombre d'erreurs, puis les lettres devinées une par une.

Attributs

Nom	Type	Rôle
wordToGuess	String	Le mot à deviner, en minuscules.
hiddenPassword	char[]	Tableau de caractères révélés ; '_' pour les lettres non trouvées.
maxChances	int	Nombre maximal d'erreurs autorisées.
remainingChances	int	Nombre d'erreurs encore possibles.
guessedLetters	Set<Character>	Ensemble des lettres déjà jouées.
startTime	long	Horodatage de début (millisecondes).
endTime	long	Horodatage de fin (0 si en cours).
finished	boolean	True quand la manche est terminée (gagnée ou perdue).

Méthodes

Signature	Description
public GameSession(String word, int maxChances)	Constructeur : initialise les attributs et démarre le chronomètre.
public boolean guess(char c)	Soumet une lettre. Renvoie true si elle est dans le mot. Décrémente remainingChances si elle n'y est pas. Une lettre déjà jouée renvoie false.
public boolean isWon()	True si toutes les positions de hiddenPassword sont révélées.
public boolean isLost()	True si remainingChances ≤ 0 et que le mot n'est pas trouvé.
public long calculateTimeScore()	Temps écoulé en millisecondes.
public long calculateScore()	Score combiné : $100 \times \text{chances restantes} + \max(0, 1000 - 10 \times \text{secondes})$. Utilisé pour le classement solo ET la mort subite multijoueur.

Signature	Description
<code>public long calculateChanceScore()</code>	Score basé uniquement sur les chances restantes (100 par chance). Utilisé pour les manches multijoueur normales.
<code>public String getDisplayWord()</code>	Renvoie le mot affiché tel que « _ a _ _ l e ».
Getters (accesseurs)	Accesseurs simples : getWordToGuess , getHiddenPassword , getMaxChances , getRemainingChances , getGuessedLetters , getStartTime , getEndTime .

Classe : Player

Package : `hangman.models`

Rôle : Représente un capitaine humain dans un match 1v1. Indépendant du classement : **matchScore** n'existe que le temps d'un match et n'est pas persisté.

Attributs

Nom	Type	Rôle
<code>alias</code>	<code>String</code>	Pseudonyme du joueur.
<code>matchScore</code>	<code>int</code>	Score cumulé sur le match 1v1 en cours.

Méthodes

Signature	Description
<code>public Player(String alias)</code>	Constructeur. <code>matchScore</code> initialisé à 0.
<code>public String getAlias()</code>	Renvoie le pseudo.
<code>public int getMatchScore()</code>	Renvoie le score courant du match.
<code>public void addMatchScore(int points)</code>	Ajoute des points au score du match.
<code>public void resetMatchScore()</code>	Remet le score à zéro (rejouer un match).
<code>public String toString()</code>	Format « alias (score) ». Utilisé dans le récap de fin de match.

Classe : ScoreRecord

Package : `hangman.models`

Rôle : DTO (Data Transfer Object) représentant une ligne de la table **Leaderboard**. Implémente **Comparable** dans l'ordre décroissant des scores.

Attributs

Nom	Type	Rôle
playerName	String	Nom du joueur.
score	long	Valeur combinée chances + temps.

Méthodes

Signature	Description
<code>public ScoreRecord(String name, long score)</code>	Constructeur.
<code>public String getPlayerName()</code>	Renvoie le nom du joueur.
<code>public long getScore()</code>	Renvoie le score.
<code>public int compareTo(ScoreRecord other)</code>	Tri naturel : score le plus élevé en premier.
<code>public String toString()</code>	Format aligné « Nom_padding Score ».

3.5 Package hangman.services

Classe : ScoreBoard

Package : hangman.services

Rôle : Couche de logique métier au-dessus de **ScoreDAO**. Les managers passent toujours par ScoreBoard, jamais directement par le DAO : cela centralise la validation (nom vide → « Anonyme ») et offre un point unique pour ajouter du cache ou de la journalisation.

Attributs

Nom	Type	Rôle
scoreDAO	ScoreDAO	DAO sous-jacent.

Méthodes

Signature	Description
public ScoreBoard()	Constructeur par défaut, instancie un ScoreDAO.
public ScoreBoard(ScoreDAO dao)	Constructeur d'injection — utile pour les tests.
public boolean isTop10(Difficulty d, long score)	Délègue à ScoreDAO.isTop10 .
public void addScore(Difficulty d, String name, long score)	Valide le nom (blanc → « Anonyme ») puis délègue à ScoreDAO.saveScore .
public List<ScoreRecord> getTop10(Difficulty d)	Délègue à ScoreDAO.getTop10Scores .

3.6 Package hangman.managers

Classe : GameManager (abstract)

Package : hangman.managers

Rôle : Classe abstraite de base pour les modes de jeu. Contient la référence à la **GameSession** en cours.

Attributs

Nom	Type	Rôle
activeSession	GameSession (protected)	Session de jeu actuelle ; peut être null entre les manches.

Méthodes

Signature	Description
public GameSession getActiveSession()	Renvoie la session active.
public abstract void startRound()	Méthode abstraite — chaque mode lance ses manches différemment.

Classe : SinglePlayerManager extends GameManager

Package : hangman.managers

Rôle : Mode Solo. Tire un mot aléatoire du dictionnaire et donne au joueur le nombre de chances défini par la difficulté.

Attributs

Nom	Type	Rôle
currentDifficulty	Difficulty	Difficulté sélectionnée.
dictionary	DictionaryDAO	DAO pour récupérer les mots.
scoreBoard	ScoreBoard	Service de classement.

Méthodes

Signature	Description
public SinglePlayerManager(Difficulty d)	Constructeur standard. Instancie un DictionaryDAO et un ScoreBoard.
public SinglePlayerManager(Difficulty d, DictionaryDAO, ScoreBoard)	Constructeur d'injection — utile pour les tests.
public Difficulty getCurrentDifficulty()	Renvoie la difficulté.
public void setCurrentDifficulty(Difficulty d)	Change la difficulté.
public void startRound()	Récupère un mot aléatoire et crée une GameSession avec le nombre de chances défini par la difficulté.
public boolean checkScoreboardEligibility()	Vérifie si le score combiné de la manche qualifie pour le top 10. Significatif uniquement après une victoire.
public void registerHighScore(String name)	Sauvegarde le score combiné dans le classement via ScoreBoard.
public ScoreBoard getScoreBoard()	Renvoie le ScoreBoard sous-jacent.

Classe : MultiplayerManager extends GameManager

Package : hangman.managers

Rôle : Mode 1v1. À chaque demi-manche, un joueur saisit un mot secret et l'autre devine. Le nombre de chances est dérivé du mot lui-même (lettres uniques, plafonné à 7). Gère aussi la mort subite si égalité.

Attributs

Nom	Type	Rôle
player1, player2	Player	Les deux joueurs.
baseDifficulty	Difficulty	Difficulté de base du match.
totalRounds	int	Nombre total de demi-manches prévues.
currentRound	int	Compteur de demi-manches déjà jouées.
isPlayer1Turn	boolean	True si c'est P1 qui choisit le mot.
isTieBreaker	boolean	True pendant la mort subite.
dictionary	DictionaryDAO	Pour tirer le mot du tiebreaker.

Méthodes

Signature	Description
public MultiplayerManager(Player p1, p2, Difficulty d, int rounds)	Constructeur. P1 choisit le mot en premier.
public boolean validateSecretWord(String word)	Vérifie que le mot ne contient que des lettres et respecte la fenêtre de longueur de la difficulté de base.
int calculateOpponentChances(String word)	Compte les lettres uniques du mot ; plafonné à 7. C'est le nombre de chances donné à l'adversaire.
public void startHalfRound(String secret)	Démarre une demi-manche : crée une GameSession avec le mot et le nombre de chances calculé.
public void switchTurn()	Incrémente le compteur de demi-manches et inverse les rôles.
public Player determinWinner()	Renvoie le joueur avec le score le plus élevé, ou null en cas d'égalité.
public void startTieBreaker()	Lance la mort subite : tire un mot et démarre une session. Délègue à pickTieBreakerWord() + startTieBreakerWithWord() .
public String pickTieBreakerWord()	Tire un mot aléatoire à la difficulté supérieure SANS démarrer de session — utilisé pour que les deux capitaines reçoivent le même mot.

Signature	Description
<pre>public void startTieBreakerWithWord(String word)</pre>	Crée une GameSession de mort subite avec un mot pré-sélectionné. Chaque appel crée une session fraîche pour que le second capitaine ne voie pas les lettres révélées du premier.
<pre>public void resetMatch()</pre>	Remet à zéro les scores et le compteur. Permet de rejouer.
Accesseurs	getPlayer1, getPlayer2, getBaseDifficulty, getTotalRounds, getCurrentRound, isPlayer1Turn, isTieBreaker, getCurrentGuesser, getCurrentChooser, isMatchOver.

3.7 Package hangman.ui

Classe : GameWindow extends Application

Package : hangman.ui

Rôle : Fenêtre conteneur principale. Possède le **Stage** et change le contenu via une **Scene** persistante (la racine est remplacée à chaque navigation pour préserver la taille de la fenêtre). Coordonne tous les écrans : menu, sélection de difficulté, jeu, configuration multijoueur, départage, classement.

Méthodes

Signature	Description
public static void main(String[] args)	Délègue à hangman.Main.main — permet de lancer aussi depuis cette classe directement dans un IDE.
public void start(Stage stage)	Point d'entrée JavaFX. Crée la Scene, attache le CSS, affiche le menu principal.
private void showMainMenu()	Affiche le menu principal : « Set Sail Alone », « Duel of Captains », « Wall of Legends », « Abandon Ship ».
private void showDifficultyPicker()	Affiche le sélecteur de difficulté pour le mode solo.
private void startSinglePlayer(Difficulty d)	Crée un SinglePlayerManager , démarre la manche, affiche la scène de jeu.
private void onSinglePlayerEnd()	Callback de fin de manche solo. Affiche le score, demande le nom si éligible au top 10, sauvegarde via le manager.
private void showPlayerSetup()	Formulaire de configuration du duel : noms, difficulté, manches.
private void promptForSecretWord()	Demande au chooser de saisir le mot secret (PasswordField).
private void onMultiplayerHalfRoundEnd()	Callback de fin de demi-manche. Calcule le score chances-only, l'ajoute au guesser, bascule le tour. Si match fini → handleMatchEnd.
private void handleMatchEnd()	Détermine le vainqueur ou lance la mort subite si égalité.
private void runTieBreaker()	Tire UN seul mot pour les DEUX capitaines, lance la première demi-manche de mort subite.
private void playTieBreakerHalf(boolean isSecondHalf, String word)	Joue une moitié de mort subite. Si c'est la première, passe la main au second capitaine avec le même mot. Si c'est la seconde, appelle handleMatchEnd.

Signature	Description
<code>private void showGameScene(String header, Runnable onEnd)</code>	Construit l'écran de jeu : fond + tuiles du mot + clavier + compteur d'erreurs + bouton quitter. Câble le clavier sur GameSession.guess .
<code>private void confirmAbandon()</code>	Boîte de dialogue de confirmation avant d'abandonner une partie.
<code>private GameSession activeSession()</code>	Renvoie la session de jeu courante quel que soit le mode.
<code>public void updateUI()</code>	Rafraîchit toutes les zones HUD : image du pendu selon le nombre d'erreurs, tuiles du mot, compteur, étiquette gagné/perdu.
<code>private void showLeaderboardPicker()</code>	Affiche le sélecteur de difficulté pour consulter le classement.
<code>private void showLeaderboard(Difficulty d)</code>	Charge et affiche le top 10 pour la difficulté donnée.
Méthodes utilitaires	menuCard, styledLabel, fieldLabel, field, bigButton, centerWrap, setRoot, showInfo, showError, styleDialog, attachCss.

Classe : BackgroundPane extends StackPane

Package : hangman.ui

Rôle : Panneau de fond qui étire une image pour remplir la fenêtre. Le contenu UI vient se poser au-dessus. Permet de changer dynamiquement l'image (utilisé pour afficher la progression du pendu).

Attributs

Nom	Type	Rôle
bgImageView	ImageView	L'ImageView lié à la taille du Pane qui affiche le fond.

Méthodes

Signature	Description
public BackgroundPane(Node content, String imagePath)	Constructeur : initialise l'ImageView, le lie aux dimensions, charge l'image et empile [fond, contenu].
public void setBackgroundImage(String imagePath)	Change dynamiquement l'image affichée.
public static Image loadImage(String resourcePath)	Charge une image en cherchant d'abord dans le classpath, puis dans le système de fichiers. Renvoie null si non trouvée.

Classe : VirtualKeyboard extends VBox

Package : hangman.ui

Rôle : Clavier QWERTY rendu sous forme de touches en bois patiné. Chaque touche est un **Button** qui déclenche un callback. Une fois jouée, la touche est désactivée et estompée.

Attributs

Nom	Type	Rôle
buttons	Map<Character, Button>	Mappe chaque lettre à son bouton, pour pouvoir le désactiver.
onLetterPressed	Consumer<Character>	Callback invoqué quand une touche est pressée.

Méthodes

Signature	Description
public VirtualKeyboard()	Constructeur : construit les 3 rangées QWERTY, attache la classe CSS key à chaque bouton, câble le callback.

Signature	Description
<code>public void setOnLetterPressed(Consumer<Character> h)</code>	Enregistre le callback à appeler à chaque pression.
<code>public void disableAndFadeButton(char c)</code>	Désactive (et estompe) une touche après son utilisation.
<code>public void resetKeyboard()</code>	Réactive toutes les touches (début d'une nouvelle manche).

Classe : MatchEndMenu extends VBox

Package : `hangman.ui`

Rôle : Carte popup affichée à la fin d'un match 1v1. Offre deux choix : « Play Again » ou « Return to Menu ». Le style visuel vient du CSS.

Méthodes

Signature	Description
<code>public MatchEndMenu(String msg, Runnable onPlayAgain, Runnable onReturn)</code>	Constructeur : construit la carte avec le titre, deux boutons, et câble les deux callbacks.

4. Système de score

Le projet utilise **deux formules de score** distinctes selon le contexte :

Formule complète (solo + mort subite)

$$\text{score} = (\text{chancesRestantes} \times 100) + \max(0, 1000 - \text{secondes} \times 10)$$

Cette formule combine deux composantes : **(a)** 100 points par chance restante en fin de manche, et **(b)** un bonus de temps qui démarre à 1000 et perd 10 points par seconde écoulée, avec un plancher à 0. Elle est utilisée :

- pour le **classement solo** (`SinglePlayerManager.registerHighScore`) ;
- pour la **mort subite multijoueur** où les deux capitaines courent contre la montre sur le même mot.

Formule sans temps (multijoueur normal)

$$\text{score} = \text{chancesRestantes} \times 100$$

Dans les manches multijoueur normales, chaque capitaine a un mot différent à résoudre ; courir contre la montre ne serait pas équitable. Seul le nombre de chances restantes compte.

Exemples

Scénario	Chances restantes	Temps (s)	Score
Win EASY parfait	10	5	$1000 + 950 = 1950$
Win EASY lent	5	60	$500 + 400 = 900$
Win HARD parfait	6	20	$600 + 800 = 1400$
Win INSANE de justesse	1	90	$100 + 100 = 200$
Défaite	0	—	0

5. Schéma de la base de données

Deux tables MySQL dans la base **hangman_db** :

Table Dictionary

Stocke le vocabulaire utilisé par le jeu, classé par difficulté.

```
CREATE TABLE Dictionary (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  word VARCHAR(64) NOT NULL,  
  difficulty ENUM('EASY', 'MEDIUM', 'HARD', 'INSANE') NOT NULL,  
  INDEX idx_difficulty (difficulty)  
);
```

Table Leaderboard

Stocke les meilleurs scores du mode solo, classés par difficulté. Plus le score est ÉLEVÉ, mieux c'est. L'index **idx_diff_score** est trié DESC pour accélérer les lectures du top 10.

```
CREATE TABLE Leaderboard (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  player_name VARCHAR(32) NOT NULL,  
  difficulty ENUM('EASY', 'MEDIUM', 'HARD', 'INSANE') NOT NULL,  
  score BIGINT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_diff_score (difficulty, score DESC)  
);
```

6. Conclusion

Le projet **Pirate's Hangman** illustre la mise en œuvre d'une architecture en couches (Modèle - Vue - Contrôleur) avec une séparation rigoureuse des responsabilités :

- les **modèles** sont passifs et ne dépendent d'aucune couche supérieure ;
- la **persistance** est entièrement encapsulée par le pattern DAO ;
- les **managers** orchestrent le déroulement des parties sans connaître les détails d'affichage ;
- la **vue JavaFX** ne contient aucune logique métier ; elle se contente de présenter l'état et de transmettre les actions utilisateur.

Cette séparation a permis d'évoluer rapidement : ajout du mode 1v1 avec mort subite, refonte complète du thème graphique, modification du système de score (deux formules selon le contexte), tout cela sans casser les couches voisines.

Le projet démontre également l'usage du **pattern Singleton (DBConnection)**, du **polymorphisme** via la classe abstraite **GameManager**, de l'**injection de dépendances** (constructeurs alternatifs des managers pour les tests), et de l'**héritage de bibliothèque** (**extends Application**, **extends VBox**, **extends StackPane**).

— Fin de la documentation —